UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Departamento de Innovación Basada en la Información y el Conocimiento

Ingeniería En Computación

PRÁCTICA 5

Descargas: Secuencial vs. Asincronía

Comparación de rendimiento entre time.sleep() y asyncio.gather()

Nombre: GONZALEZ MARTINEZ ALONDRA GORETTI	**Código**: 218534681
Materia: Programación paralela y concurrente	**Grupo**: _____
Nombre del profesor: Mtro. Zeus Cobián	**Actividad**: Práctica 5 (100 puntos)
Fecha de entrega: 6 de noviembre del 2026	**Lenguaje**: Python 3.12 · asyncio

Bibliografía:

Python Software Foundation. (s. f.). *asyncio — Asynchronous I/O*. Python 3 Documentation.

Python Software Foundation. (s. f.). *Coroutines and tasks*. Python 3 Documentation.

Selivanov, Y. (2015). *PEP 492 – Coroutines with async and await syntax*.

Recursos didácticos de la asignatura (Simultaneidad y asincronía).

1. Objetivo de la práctica

Aplicar programación asincrónica en Python para simular la descarga de cuatro archivos con duración aleatoria y comparar el rendimiento entre la ejecución secuencial (bloqueante) y la asincrónica (no bloqueante) usando `async`, `await` y `asyncio.gather()`, registrando el tiempo total de cada enfoque.

2. Fundamento breve

En la ejecución **secuencial** cada descarga espera a que termine la anterior: `time.sleep(d)` detiene todo el programa durante *d* segundos y el procesador queda ocioso. En la ejecución **asincrónica**, una función definida con `async def` es una corrutina: al llegar a un `await` se suspende y devuelve el control al *bucle de eventos* de `asyncio`, que aprovecha para avanzar las demás (Selivanov, 2015). `asyncio.gather()` programa todas las corrutinas a la vez y regresa cuando la última termina (Python Software Foundation, s. f.-b). Con descargas independientes, el modelo ideal predice que el tiempo secuencial es la **suma** de las duraciones y el asincrónico es la **mayor** de ellas.

Metodología de medición

- Se sortean 4 duraciones entre 1 y 4 s con `random.Random(semilla)` y se usan **las mismas** en ambos enfoques.
- Se mide solo la ejecución de cada bloque con `time.perf_counter()`.
- Corrida de referencia con semilla 2025 y, después, 10 corridas con duraciones distintas y un experimento con 1 a 32 archivos.
- Entorno de prueba: Python 3.12.3; los tiempos absolutos dependen del equipo, pero la relación suma/máximo se conserva.

3. Código de la práctica

3.1 Programa completo (descargas.py)

descargas.py · Python 3

```python
"""Práctica 5 · Simultaneidad y Asincronía.

Simula la descarga de 4 archivos con duración aleatoria y compara:
  1) Ejecución secuencial (bloqueante)      -> time.sleep()
  2) Ejecución asincrónica (no bloqueante)  -> async / await / asyncio.gather()

Uso:   python descargas.py            (duraciones aleatorias distintas en cada corrida)
       python descargas.py --semilla 7 (mismas duraciones en ambos bloques y reproducibles)

Nota para Google Colab / Jupyter: ahí ya existe un bucle de eventos en marcha, por lo
que en lugar de asyncio.run(...) se escribe:   await main_asincrono(duraciones)
"""
import argparse
import asyncio
import random
import time

ARCHIVOS = ["video.mp4", "documento.pdf", "imagen.png", "datos.csv"]


```

```python
def generar_duraciones(n=4, minimo=1.0, maximo=4.0, semilla=None):
    """Duración (s) aleatoria para cada archivo. Se generan UNA sola vez y se usan en
    ambos enfoques, para que la comparación sea justa."""
    rng = random.Random(semilla)
    return [round(rng.uniform(minimo, maximo), 2) for _ in range(n)]


# ---------------------------------------------------------------------------- #
# 1) BLOQUE SECUENCIAL (bloqueante)
# ---------------------------------------------------------------------------- #
def descargar_sync(nombre, duracion, t0, eventos, verbose=True):
    inicio = time.perf_counter() - t0
    if verbose:
        print(f"[{inicio:6.2f} s] Iniciando  {nombre} (durará {duracion:.2f} s)")
    time.sleep(duracion)                    # BLOQUEA: nada más puede ejecutarse
    fin = time.perf_counter() - t0
    if verbose:
        print(f"[{fin:6.2f} s] Terminó     {nombre}")
    eventos.append((nombre, inicio, fin, duracion))


def ejecutar_secuencial(duraciones, nombres=None, verbose=True):
    nombres = nombres or [f"archivo_{i + 1}" for i in range(len(duraciones))]
    eventos = []
    t0 = time.perf_counter()
    for nombre, dur in zip(nombres, duraciones):
        descargar_sync(nombre, dur, t0, eventos, verbose)
    return time.perf_counter() - t0, eventos


# ---------------------------------------------------------------------------- #
# 2) BLOQUE ASINCRÓNICO (no bloqueante)
# ---------------------------------------------------------------------------- #
async def descargar_async(nombre, duracion, t0, eventos, verbose=True):
    inicio = time.perf_counter() - t0
    if verbose:
        print(f"[{inicio:6.2f} s] Iniciando  {nombre} (durará {duracion:.2f} s)")
    await asyncio.sleep(duracion)           # CEDE el control: otras descargas avanzan
```

```
    fin = time.perf_counter() - t0
    if verbose:
        print(f"[{fin:6.2f} s] Terminó     {nombre}")
    eventos.append((nombre, inicio, fin, duracion))


async def main_asincrono(duraciones, nombres=None, verbose=True):
    nombres = nombres or [f"archivo_{i + 1}" for i in range(len(duraciones))]
    eventos = []
    t0 = time.perf_counter()
    await asyncio.gather(*(descargar_async(n, d, t0, eventos, verbose)
                           for n, d in zip(nombres, duraciones)))
    return time.perf_counter() - t0, eventos


def ejecutar_asincrono(duraciones, nombres=None, verbose=True):
    return asyncio.run(main_asincrono(duraciones, nombres, verbose))


# ---------------------------------------------------------------------------- #
def main():
    ap = argparse.ArgumentParser(description="Práctica 5: secuencial vs asincrónico")
    ap.add_argument("--semilla", type=int, default=None, help="semilla para reproducir las duraciones")
    args = ap.parse_args()

    duraciones = generar_duraciones(len(ARCHIVOS), semilla=args.semilla)
    print("Duraciones simuladas:", dict(zip(ARCHIVOS, duraciones)))
    print(f"Suma de duraciones: {sum(duraciones):.2f} s | Mayor duración: {max(duraciones):.2f} s\n")

    print("=== BLOQUE 1: ejecución SECUENCIAL (bloqueante) ===")
    t_sec, _ = ejecutar_secuencial(duraciones, ARCHIVOS)
    print(f"--> Tiempo total secuencial: {t_sec:.2f} s\n")

    print("=== BLOQUE 2: ejecución ASINCRÓNICA (asyncio.gather) ===")
    t_asy, _ = ejecutar_asincrono(duraciones, ARCHIVOS)
    print(f"--> Tiempo total asincrónico: {t_asy:.2f} s\n")

    print("=== TABLA COMPARATIVA ===")
```

```
    print(f"{'Enfoque':<22}{'Tiempo total (s)':>18}")
    print(f"{'Secuencial':<22}{t_sec:>18.2f}")
    print(f"{'Asincrónico':<22}{t_asy:>18.2f}")
    print(f"{'Aceleración':<22}{t_sec / t_asy:>17.2f}x")
    print(f"{'Tiempo ahorrado':<22}{t_sec - t_asy:>16.2f} s")


if __name__ == "__main__":
    main()
```

El script *experimento.py* (en el repositorio) repite las mediciones y genera las tablas y gráficas de la sección 4.

4. Resultados

4.1 Capturas de la ejecución

Salida real del programa con la semilla 2025 (se muestran la tabla comparativa y el inicio/fin de cada descarga):

terminal — descargas.py

```
$ python descargas.py --semilla 2025
Duraciones simuladas: {'video.mp4': 2.67, 'documento.pdf': 2.94, 'imagen.png': 2.43, 'datos.csv': 1.52}
Suma de duraciones: 9.56 s | Mayor duración: 2.94 s

=== BLOQUE 1: ejecución SECUENCIAL (bloqueante) ===
[  0.00 s] Iniciando  video.mp4 (durará 2.67 s)
[  2.67 s] Terminó     video.mp4
[  2.67 s] Iniciando  documento.pdf (durará 2.94 s)
[  5.61 s] Terminó     documento.pdf
[  5.61 s] Iniciando  imagen.png (durará 2.43 s)
[  8.04 s] Terminó     imagen.png
[  8.04 s] Iniciando  datos.csv (durará 1.52 s)
[  9.56 s] Terminó     datos.csv
--> Tiempo total secuencial: 9.56 s

=== BLOQUE 2: ejecución ASINCRÓNICA (asyncio.gather) ===
[  0.00 s] Iniciando  video.mp4 (durará 2.67 s)
[  0.00 s] Iniciando  documento.pdf (durará 2.94 s)
[  0.00 s] Iniciando  imagen.png (durará 2.43 s)
[  0.00 s] Iniciando  datos.csv (durará 1.52 s)
[  1.52 s] Terminó     datos.csv
[  2.43 s] Terminó     imagen.png
[  2.67 s] Terminó     video.mp4
[  2.94 s] Terminó     documento.pdf
--> Tiempo total asincrónico: 2.94 s

=== TABLA COMPARATIVA ===
Enfoque                Tiempo total (s)
Secuencial                         9.56
Asincrónico                        2.94
Aceleración                       3.25x
Tiempo ahorrado                6.62 s
```

Figura 1. Ejecución del bloque secuencial y del asincrónico. *Fuente: elaboración propia.*

4.2 Comparación

Tabla 1. Tiempo total de la corrida de referencia y aceleración de la versión asincrónica.

Enfoque	Suma de duraciones (s)	Tiempo total (s)	Aceleración
Secuencial (bloqueante)	9.56	9.56	**1.00×**
Asincrónico (gather)	2.94 (mayor)	2.94	**3.25×**

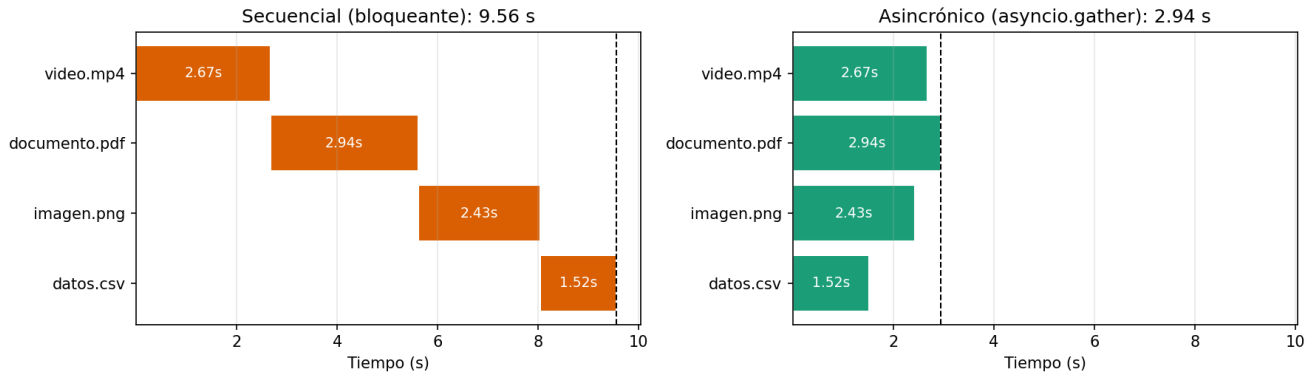


Figura 2. Línea de tiempo de la corrida de referencia (la línea punteada marca el tiempo total). *Fuente: elaboración propia.*

Tabla 2. Repeticiones con duraciones aleatorias distintas (s).

Corrida	Duraciones	Suma	Mayor	Secuencial	Asincrónico	Aceleración
1	1.36 / 2.51 / 2.54 / 3.58	9.99	3.58	9.99	3.58	**2.79×**
2	1.32 / 2.39 / 1.04 / 1.97	6.72	2.39	6.72	2.39	**2.81×**
3	2.07 / 1.54 / 1.08 / 2.09	6.78	2.09	6.78	2.09	**3.24×**
4	3.16 / 2.45 / 3.77 / 3.23	12.61	3.77	12.61	3.77	**3.34×**
5	1.86 / 2.12 / 2.50 / 2.41	8.89	2.50	8.89	2.50	**3.56×**
6	2.99 / 2.00 / 3.32 / 3.35	11.66	3.35	11.66	3.35	**3.48×**
7	3.48 / 2.75 / 3.36 / 2.97	12.56	3.48	12.56	3.48	**3.61×**
8	2.23 / 3.86 / 2.18 / 2.68	10.95	3.86	10.95	3.86	**2.84×**
9	1.23 / 2.08 / 3.42 / 1.15	7.88	3.42	7.88	3.42	**2.30×**
10	3.44 / 3.38 / 3.83 / 1.10	11.75	3.83	11.75	3.83	**3.07×**
Promedio		**9.98**	**3.23**	**9.98**	**3.23**	**3.10×**

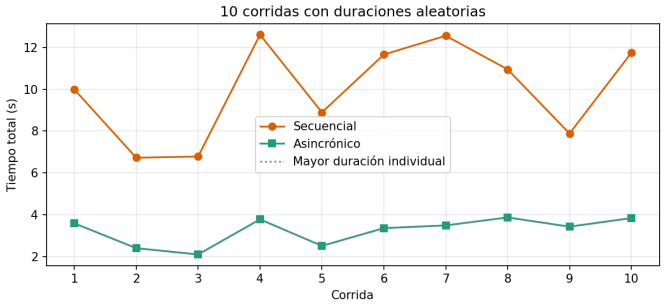


Figura 3. Tiempo total en cada corrida; el asincrónico sigue a la mayor duración individual. *Fuente: elaboración propia.*

Tabla 3. Tiempo total según el número de descargas (0.1–0.5 s cada una).

Archivos	Secuencial (s)	Asincrónico (s)	Aceleración
1	0.320	0.321	**1.00×**
2	0.681	0.361	**1.89×**
4	1.141	0.361	**3.16×**
8	2.192	0.491	**4.47×**
16	4.143	0.491	**8.43×**
32	9.357	0.492	**19.01×**

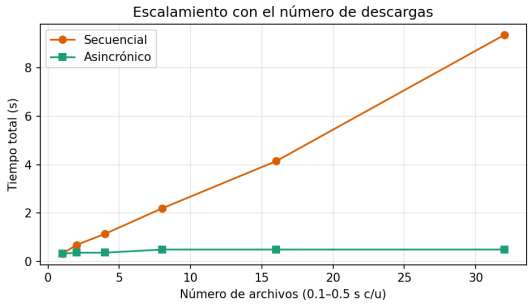


Figura 4. Escalamiento: el tiempo secuencial crece con el número de archivos; el asincrónico casi no cambia. *Fuente: elaboración propia.*

Nota: la práctica *simula* las descargas con esperas; en una descarga real el ancho de banda compartido puede limitar la ganancia, y entonces conviene acotar la concurrencia con `asyncio.Semaphore`. Los tiempos varían unos milisegundos entre ejecuciones, pero la relación suma/máximo se mantiene siempre.

5. Reflexión final

¿Qué ventaja representa la asincronía frente a la ejecución secuencial? Aprovecha el tiempo de espera: mientras una descarga aguarda, el bucle de eventos avanza las demás, por lo que el tiempo total deja de ser la suma de todas las esperas y pasa a ser aproximadamente la más larga. En la corrida de referencia se pasó de 9.56 s a 2.94 s (3.25 veces más rápido, 69 % menos tiempo) sin crear hilos ni procesos adicionales y con un código que se sigue leyendo de forma lineal. La ventaja crece con el número de tareas (casi 19 veces con 32 archivos); su límite es que solo ayuda cuando el programa espera entrada/salida, no cuando hace cálculo intensivo de CPU.

¿En qué tipo de programas reales sería útil esta técnica? En programas dominados por esperas de red o de disco: servidores web y APIs que atienden muchos clientes a la vez (FastAPI, aiohttp), rastreadores web que consultan cientos de páginas, descargadores de archivos, bots de chat, aplicaciones que consultan varias APIs o bases de datos para armar una sola respuesta, y sistemas de tiempo real como chats, notificaciones o lectura de sensores.

6. Conclusión

La versión **asincrónica fue la más rápida**: 2.94 s frente a 9.56 s en la corrida de referencia y 3.23 s frente a 9.98 s en promedio en 10 corridas. Los resultados confirman el modelo teórico (suma contra máximo): el tiempo secuencial coincidió con la suma de las duraciones (9.56 s) y el asincrónico con la mayor (2.94 s), con apenas unos milisegundos de sobrecarga. Además, la mejora aumenta con el número de tareas independientes y desaparece con una sola tarea o con trabajo de CPU. `async`, `await` y `asyncio.gather()` permiten obtener esa ganancia con un código corto y legible.

Referencias

Python Software Foundation. (s. f.-a). *asyncio — Asynchronous I/O*. Python 3 Documentation. https://docs.python.org/3/library/asyncio.html

Python Software Foundation. (s. f.-b). *Coroutines and tasks*. Python 3 Documentation. https://docs.python.org/3/library/asyncio-task.html

Selivanov, Y. (2015). *PEP 492 – Coroutines with async and await syntax*. Python Enhancement Proposals. https://peps.python.org/pep-0492/